

Visual Studio Code para análise de imagens

Instalação, extensões, interface, Python, notebooks e ambientes Pixi

NanoCell Interactions Lab

2026-07-27

Índice

1	1. Instalar o VS Code	3
1.1	1.1. Habilitar o comando code	3
2	2. Instalar as extensões usadas no curso	4
2.1	2.1. Extensões essenciais	4
2.2	2.2. Extensões auxiliares	5
3	3. Conhecer a interface	6
3.1	3.1. Mãos na massa — Explorando a interface	7
4	4. Usar a Paleta de Comandos	7
4.1	4.1. Mãos na massa — Comandos de edição	8
5	5. Criar e executar um arquivo Python	9
5.1	5.1. Selecionar o interpretador Python	10
5.2	5.2. Executar o arquivo	10
6	6. Trabalhar com um projeto Pixi	10
6.1	6.1. Instalar o ambiente	11
6.2	6.2. Abrir o projeto	11
6.3	6.3. Identificar o ambiente no VS Code	11
7	7. Trabalhar com notebooks Jupyter	12
7.1	7.1. Testar os pacotes	12
8	8. Entender o terminal integrado	13
8.1	8.1. Usar pixi shell	14

9	9. Formatação, análise estática e organização do código	15
9.1	9.1. Black Formatter	15
9.2	9.2. isort	16
9.3	9.3. Pylint e análise estática	16
10	10. Configurar réguas de comprimento de linha	17
11	11. Desafio integrado	18
11.1	11.1 Parte 1 — Criar o script	18
11.2	11.2 Parte 2 — Explorar o VS Code	19
11.3	11.3 Parte 3 — Criar um notebook	19
12	12. Checklist antes de iniciar uma análise	19
13	13. Problemas frequentes	20
14	14. Mensagens principais	21
15	15 Para aprofundar	21

O **Visual Studio Code**, ou VS Code, será uma das ferramentas mais utilizadas ao longo deste curso. Ele permite trabalhar, no mesmo programa, com:

- arquivos de texto e Markdown;
- scripts Python;
- notebooks Jupyter;
- arquivos de configuração, como `pixi.toml`;
- tabelas e outros arquivos do projeto;
- terminal;
- controle de versão com Git.

O VS Code é um editor extensível. Isso significa que novas funções podem ser adicionadas por meio de extensões, também chamadas de *plugins*. Com as extensões adequadas, ele se torna um ambiente integrado para editar, executar, revisar e organizar análises computacionais.

i VS Code é um IDE?

O VS Code é frequentemente usado como um ambiente de desenvolvimento integrado, ou IDE, embora sua instalação básica seja mais próxima de um editor de código. As extensões adicionam recursos como execução de Python, notebooks, análise estática, formatação, depuração e integração com ambientes computacionais.

! Objetivo deste tutorial

O objetivo não é explorar todos os recursos do VS Code. Ao final, você deverá conseguir:

1. abrir uma pasta de projeto;
2. localizar e editar arquivos;
3. instalar as extensões usadas no curso;
4. usar a Paleta de Comandos;
5. abrir o terminal integrado;
6. selecionar o interpretador Python;
7. selecionar o kernel de um notebook;
8. trabalhar com um ambiente Pixi;
9. criar e executar um arquivo Python simples.

1 1. Instalar o VS Code

Baixe o instalador no site oficial:

[Baixar o Visual Studio Code](#)

Escolha a versão correspondente ao seu sistema operacional e siga as instruções de instalação.

Depois de instalar, abra o programa para confirmar que ele inicia normalmente.

1.1 1.1. Habilitar o comando `code`

O comando:

```
code .
```

abre no VS Code a pasta em que o terminal está localizado. O ponto (.) significa a pasta atual.

1.1.1 macOS

1. abra o VS Code;
2. abra a Paleta de Comandos com `⇧⌘P`;
3. procure por:

```
Shell Command: Install 'code' command in PATH
```

4. execute o comando;
5. feche e reabra o terminal.

1.1.2 Windows

A instalação normalmente adiciona o comando code ao PATH. Caso ele não funcione, feche e reabra o terminal. Se necessário, execute novamente o instalador e verifique a opção para adicionar o VS Code ao PATH.

1.1.3 Teste

No terminal, execute:

```
code --version
```

Se uma versão for exibida, o comando está disponível.

2 2. Instalar as extensões usadas no curso

Abra o painel de extensões usando:

- Windows/Linux: Ctrl+Shift+X;
- macOS: ⌘+X;

ou vá em:

```
View > Extensions
```

Pesquise o nome de cada extensão e confira também o nome do publicador antes de instalar.

2.1 2.1. Extensões essenciais

Instale primeiro estas extensões:

Extensão	Publicador	Função principal
Python	Microsoft	Suporte à linguagem Python, execução, depuração e seleção de interpretadores
Pylance	Microsoft	Autocompletar, navegação pelo código, análise de tipos e ajuda durante a escrita
Jupyter	Microsoft	Abertura e execução de notebooks .ipynb
Pixi Code	Renan R. Santos	Integração de ambientes Pixi com o VS Code
Black Formatter	Microsoft	Formatação automática e consistente de arquivos Python
isort	Microsoft	Organização automática das instruções import
Pylint	Microsoft	Análise estática e identificação de problemas potenciais no código

Instalação automática de dependências

Algumas extensões podem sugerir ou instalar extensões relacionadas. Por exemplo, a extensão Python pode recomendar o Pylance. Antes de instalar novamente, confira se a extensão já aparece como instalada.

2.2 2.2. Extensões auxiliares

Estas extensões não são indispensáveis para executar o curso, mas podem facilitar a escrita:

Extensão	Publicador	Utilidade
Python Snippets	Ferhat Yalçın	Atalhos para inserir estruturas comuns de Python
autoDocstring — Python Docstring Generator	Nils Werner	Geração de modelos de documentação para funções

Extensão	Publicador	Utilidade
Python Indent	Kevin Rose	Auxílio com indentação em estruturas Python

Extensões essenciais e opcionais

Comece pelas extensões essenciais. Instalar muitas extensões de uma vez pode aumentar a complexidade da interface e dificultar a identificação da origem de avisos ou comportamentos inesperados.

3 3. Conhecer a interface

O VS Code organiza o trabalho em torno de uma pasta de projeto, também chamada de *workspace*. Sempre que possível, abra a pasta inteira do projeto, e não apenas um arquivo isolado.

A barra lateral esquerda é chamada de Activity Bar. Os principais painéis que utilizaremos são:

1. Explorer — mostra arquivos e pastas do projeto;
2. Search — procura palavras ou trechos em todos os arquivos;
3. Source Control — mostra alterações controladas pelo Git;
4. Run and Debug — executa e depura código;
5. Extensions — instala e gerencia extensões;
6. Testing — executa testes quando o projeto possui uma estrutura de testes.

Na parte inferior da janela, encontramos o Panel, que pode conter:

- terminal;
- problemas detectados;
- saída de extensões;
- console de depuração.

A barra inferior é chamada de Status Bar. Dependendo do arquivo aberto, ela pode mostrar:

- interpretador Python selecionado;
- branch do Git;
- codificação do arquivo;

- tipo de quebra de linha;
- linguagem do arquivo;
- mensagens relacionadas ao ambiente.

3.1 Mãos na massa — Explorando a interface

1. Crie uma pasta chamada `tutorial-vscode`.
2. Abra o VS Code.
3. Vá em `File > Open Folder`.
4. Selecione a pasta criada.
5. No Explorer, crie:
 - uma pasta chamada `dados`;
 - um arquivo chamado `README.md`;
 - um arquivo chamado `boas_vindas.py`.
6. Use o painel Search para procurar a palavra `boas`.
7. Abra e feche o terminal integrado.

A estrutura deverá ficar assim:

```
tutorial-vscode/  
├─ dados/  
├─ README.md  
└─ boas_vindas.py
```

Confirme a pasta aberta

O nome da pasta do projeto aparece no Explorer. Antes de criar arquivos ou executar comandos, confirme que o VS Code está aberto na pasta correta.

4 4. Usar a Paleta de Comandos

A Paleta de Comandos permite localizar funções sem precisar memorizar em qual menu elas estão.

Abra a paleta usando:

- Windows/Linux: `Ctrl+Shift+P`;

- macOS: ⌘P ;
- ou pressione F1.

4.1 Mãos na massa — Comandos de edição

No arquivo `README.md`, escreva:

```
microscopia  
python  
imagem  
python  
pixi
```

Agora experimente:

4.1.1 4.1. Transformar texto em maiúsculas

1. selecione uma palavra;
2. abra a Paleta de Comandos;
3. procure por:

```
Transform to Uppercase
```

4.1.2 4.2. Ordenar linhas

1. selecione todas as linhas;
2. abra a Paleta de Comandos;
3. procure por:

```
Sort Lines Descending
```

4.1.3 4.3. Criar múltiplos cursores

Adicione cursores em várias linhas usando:

- Windows/Linux: $\text{Alt} + \text{clique}$;

- macOS: Option (⌘) + clique.

Digite simultaneamente no início das linhas:

```
item =
```

Múltiplos cursores permitem repetir a mesma edição em vários locais.

4.1.4 4.4. Selecionar ocorrências iguais

Posicione o cursor sobre uma das ocorrências de python e use:

- Windows/Linux: Ctrl+D;
- macOS: ⌘D.

Pressione o atalho novamente para selecionar a próxima ocorrência. Depois, edite as palavras selecionadas ao mesmo tempo.

i Nem todos os atalhos são iguais entre sistemas

Atalhos podem variar conforme o sistema operacional, o teclado e personalizações locais. A Paleta de Comandos é uma alternativa confiável quando um atalho não funciona.

5 5. Criar e executar um arquivo Python

Abra boas_vindas.py e escreva:

```
nome = "HCI/HCA"

print(f"Bem-vindo ao curso de {nome}!")
```

Salve o arquivo usando:

- Windows/Linux: Ctrl+S;
- macOS: ⌘S.

5.1 5.1. Selecionar o interpretador Python

Abra a Paleta de Comandos e procure:

```
Python: Select Interpreter
```

Escolha o interpretador associado ao ambiente do projeto.

O interpretador selecionado é usado por recursos como:

- execução de scripts;
- depuração;
- autocompletar;
- análise do código;
- formatação e linting, dependendo da configuração.

5.2 5.2. Executar o arquivo

Com o arquivo aberto, você pode usar o botão de execução no canto superior direito ou abrir o terminal e executar:

```
python boas_vindas.py
```

Quando estiver trabalhando com Pixi, a forma mais explícita é:

```
pixi run python boas_vindas.py
```

A saída deverá ser:

```
Bem-vindo ao curso de HCI/HCA!
```

6 6. Trabalhar com um projeto Pixi

Em um projeto Pixi, a pasta principal geralmente contém:

```
projeto/  
├─ pixi.toml  
├─ pixi.lock  
├─ notebooks/  
├─ dados/  
└─ scripts/
```

O arquivo `pixi.toml` descreve o ambiente e suas dependências. O arquivo `pixi.lock` registra as versões resolvidas para tornar a execução mais reproduzível.

6.1 6.1. Instalar o ambiente

No terminal, entre na pasta que contém o `pixi.toml` e execute:

```
pixi install
```

Se o `pixi.toml` já existe, não é necessário executar `pixi init`.

6.2 6.2. Abrir o projeto

Na mesma pasta, execute:

```
code .
```

Não é obrigatório ativar o ambiente com `pixi shell` antes de abrir o VS Code.

6.3 6.3. Identificar o ambiente no VS Code

Com as extensões Python e Pixi Code instaladas:

1. abra a Paleta de Comandos;
2. execute `Python: Select Interpreter`;
3. procure o Python associado ao projeto Pixi;
4. selecione-o.

O caminho costuma estar dentro da pasta `.pixi`, embora a forma exata de exibição possa variar.

! Abrir com code . não escolhe automaticamente o kernel

Abrir o VS Code a partir de um terminal com o ambiente ativado pode ajudar algumas ferramentas a encontrar executáveis, mas isso não garante que um notebook usará o ambiente correto.

Sempre confira explicitamente:

- o interpretador, para arquivos `.py`;
- o kernel, para notebooks `.ipynb`.

7 7. Trabalhar com notebooks Jupyter

Abra um arquivo `.ipynb` ou crie um novo notebook pela Paleta de Comandos:

Create: New Jupyter Notebook

No canto superior direito do notebook, clique em:

Select Kernel

Escolha o ambiente Python correspondente ao projeto Pixi.

Execute uma célula de teste:

```
import sys

print(sys.executable)
print(sys.version)
```

O caminho mostrado em `sys.executable` deve corresponder ao ambiente pretendido.

7.1 7.1. Testar os pacotes

Execute:

```
import matplotlib
import numpy
import pandas

print("Ambiente pronto.")
```

Se a célula for executada sem erros, esses pacotes estão disponíveis no kernel selecionado.

 Interpretador e kernel não são exatamente a mesma seleção

O interpretador selecionado para arquivos Python e o kernel selecionado para notebooks podem aparecer em listas diferentes. É possível que um script esteja configurado para um ambiente e o notebook esteja executando em outro.

Confira o kernel no canto superior direito de cada notebook.

8. Entender o terminal integrado

Abra o terminal em:

```
Terminal > New Terminal
```

ou use:

- Windows/Linux: Ctrl+`;
- macOS: Control+`.

O terminal integrado funciona como um terminal normal, apenas exibido dentro do VS Code.

 O terminal não é o mesmo que o interpretador ou o kernel

Ter um ambiente selecionado no VS Code não garante que todos os comandos digitados em um terminal já aberto usem esse ambiente.

Antes de instalar pacotes ou executar uma análise, confirme onde o comando será executado.

Para verificar qual Python está disponível no terminal:

8.0.1 macOS/Linux/Git Bash

```
which python  
python --version
```

8.0.2 Windows PowerShell ou Prompt de Comando

```
where.exe python  
python --version
```

Com Pixi, prefira executar comandos de forma explícita:

```
pixi run python --version  
pixi run python script.py  
pixi run notebook
```

Dessa forma, não é necessário depender da ativação manual do ambiente.

8.1 8.1. Usar `pixi shell`

Caso queira ativar o ambiente no terminal:

```
pixi shell
```

Depois, verifique:

```
python --version
```

Para sair:

```
exit
```

 Quando usar cada abordagem?

Use `pixi run` quando quiser executar uma tarefa de forma explícita e reproduzível:

```
pixi run python script.py
```

Use `pixi shell` quando quiser executar vários comandos consecutivos dentro do ambiente:

```
pixi shell
python script_1.py
python script_2.py
exit
```

9 9. Formatação, análise estática e organização do código

9.1 9.1. Black Formatter

O Black organiza automaticamente o estilo do código Python.

Para formatar um arquivo:

1. abra a Paleta de Comandos;
2. procure por:

Format Document

3. caso seja solicitado, escolha Black Formatter.

Para tornar o Black o formatador padrão:

1. abra um arquivo `.py`;
2. abra a Paleta de Comandos;
3. execute:

```
Format Document With...
```

4. selecione:

```
Configure Default Formatter
```

5. escolha Black Formatter.

9.2 9.2. isort

O isort organiza as instruções de importação.

Exemplo antes da organização:

```
import pandas as pd
from pathlib import Path
import numpy as np
```

Depois da organização:

```
from pathlib import Path

import numpy as np
import pandas as pd
```

Use a Paleta de Comandos e procure por:

```
Organize Imports
```

9.3 9.3. Pylint e análise estática

Um analisador estático examina o código sem precisar executar todas as suas instruções. Ele pode indicar:

- nomes não definidos;

- imports não utilizados;
- variáveis não utilizadas;
- problemas de estilo;
- construções potencialmente problemáticas.

O Pylint pode sublinhar trechos e enviar mensagens para o painel:

View > Problems

i Aviso não significa necessariamente erro de execução

Uma linha destacada pelo Pylint pode continuar sendo executada. A mensagem deve ser lida e avaliada: algumas indicam erros reais; outras são recomendações de estilo ou manutenção.

10 10. Configurar régua de comprimento de linha

Réguas verticais ajudam a perceber quando uma linha ficou muito longa.

Abra as configurações usando:

- Windows/Linux: `Ctrl+,`;
- macOS: `⌘,`.

Procure por:

`editor.rulers`

Ou abra as configurações em JSON:

Preferences: Open User Settings (JSON)

Adicione:

```
"editor.rulers": [80, 120]
```

Isso cria régua nas colunas 80 e 120.

Para ajustar a cor:

```
"workbench.colorCustomizations": {  
    "editorRuler.foreground": "#ff4081"  
}
```

💡 Configuração de usuário ou do projeto?

- User Settings afetam todos os projetos abertos pelo usuário.
- Workspace Settings afetam apenas o projeto atual e podem ser salvas em `.vscode/settings.json`.

Configurações específicas do curso ou do pipeline podem ser mantidas no workspace para que o comportamento seja mais consistente entre participantes.

11 11. Desafio integrado

Use o projeto `tutorial-vscode` criado anteriormente.

11.1 Parte 1 — Criar o script

Substitua o conteúdo de `boas_vindas.py` por:

```
from pathlib import Path  
  
curso = "HCI/HCA"  
arquivos = ["imagem.tif", "medidas.csv", "notebook.ipynb"]  
  
pasta = Path("dados")  
  
print(f"Curso: {curso}")  
print(f"Pasta de dados: {pasta.resolve()}")  
  
for arquivo in arquivos:  
    print(f"- {arquivo}")
```

11.2 Parte 2 — Explorar o VS Code

1. selecione o interpretador Python;
2. formate o documento com Black;
3. organize os imports;
4. abra o painel Problems;
5. execute usando:

```
pixi run python boas_vindas.py
```

caso o projeto tenha um ambiente Pixi; ou:

```
python boas_vindas.py
```

caso esteja utilizando outro ambiente já configurado.

11.3 Parte 3 — Criar um notebook

Crie `teste_ambiente.ipynb` e execute:

```
import sys
from pathlib import Path

print(sys.executable)
print(Path.cwd())
```

Compare:

- o caminho do kernel;
- a pasta de trabalho;
- o caminho mostrado pelo terminal.

12 12. Checklist antes de iniciar uma análise

Antes de executar um notebook ou pipeline, confirme:

- ☐ abri a pasta correta do projeto;
- ☐ o `pixi.toml` está na pasta esperada;
- ☐ executei `pixi install`;
- ☐ selecionei o interpretador correto;
- ☐ selecionei o kernel correto no notebook;
- ☐ conferi o caminho exibido por `sys.executable`;
- ☐ sei em qual pasta o terminal está;
- ☐ estou usando `pixi run` ou um shell Pixi quando necessário;
- ☐ não estou instalando pacotes acidentalmente no ambiente base ou no Python global.

13 13. Problemas frequentes

Problema	Possível causa	O que verificar
<code>code: command not found</code>	O comando <code>code</code> não está no PATH	Instale o comando pela Paleta no macOS ou reabra/reinstale no Windows
O Python não aparece	Ambiente ainda não instalado ou não detectado	Execute <code>pixi install</code> , recarregue a janela e selecione o interpretador
O notebook fica carregando indefinidamente	Kernel incorreto, <code>ipykernel</code> ausente ou ambiente inconsistente	Confira <code>Select Kernel</code> e teste <code>pixi run python -c "import ipykernel"</code>
O script funciona, mas o notebook não	Interpretador e kernel diferentes	Confira o kernel no canto superior direito
Um pacote não é encontrado	O código foi executado em outro ambiente	Verifique <code>sys.executable</code> e use <code>pixi run</code>
O terminal mostra (base)	Conda ativou automaticamente o ambiente base	Não use <code>conda install</code> ; execute pelo Pixi ou abra um novo terminal
A extensão não encontra o ambiente	A pasta aberta não é a raiz do projeto	Abra a pasta que contém <code>pixi.toml</code>
O VS Code mostra muitos avisos	Linter, analisador de tipos ou extensões estão ativos	Leia o painel Problems e diferencie erro de recomendação

Para recarregar o VS Code sem fechar o projeto:

1. abra a Paleta de Comandos;
2. execute:

```
Developer: Reload Window
```

14 14. Mensagens principais

O VS Code organiza arquivos, código, notebooks, extensões e terminal em uma única interface.

Abrir o projeto com `code .` não substitui a seleção do interpretador e do kernel.

O interpretador executa scripts Python; o kernel executa células de notebooks; o terminal executa os comandos digitados nele.

Em projetos Pixi, `pixi run` é a forma mais explícita de garantir que um comando seja executado no ambiente correto.

Não confie apenas na aparência do nome do ambiente. Verifique o caminho usando `sys.executable`, `which python` ou `where.exe python`.

15 Para aprofundar

- [Documentação oficial do Visual Studio Code](#)
- [Python no Visual Studio Code](#)
- [Ambientes Python no Visual Studio Code](#)
- [Notebooks Jupyter no Visual Studio Code](#)
- [Gerenciamento de kernels Jupyter](#)
- [Interface de linha de comando do VS Code](#)
- [Pixi Code no Visual Studio Marketplace](#)